

Precision Timing & Skill Check

*Versão : **1.0***
Desenvolvido por : @sazoncomcafe
Data Última Atualização: 10/10/2026

Introdução

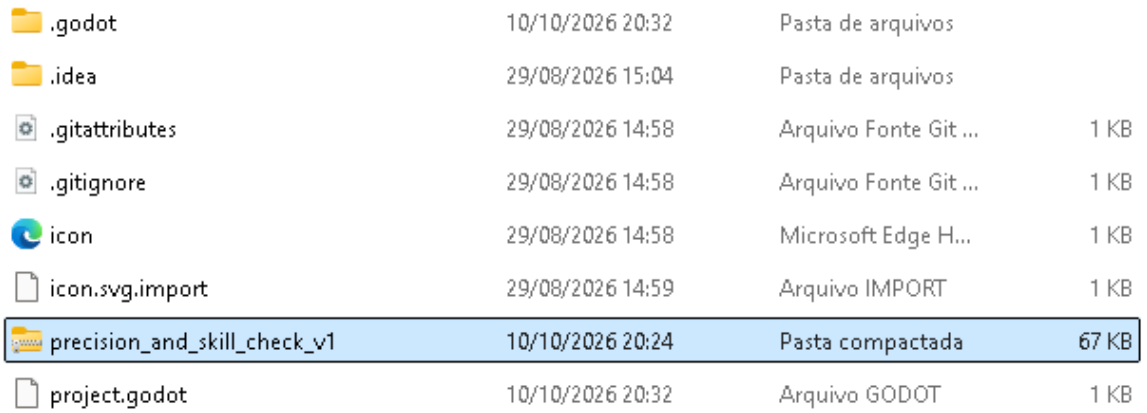
Inspirado nos mais diversos Jogos sejam Indies ou AAA com recursos já conhecidos. De fácil manipulação da Godot Engine nas versão 4.x em diante, tendo sido desenvolvido na Godot 4.6.2 stable. Aqui você irá encontrar tudo necessário para a manipulação deste recurso para a facilitação da criação do seu minigame em seu jogo. Este recurso tentou seguir ao máximo os princípios de *SOLID* e *Arquitetura do Observer*

Objetivo

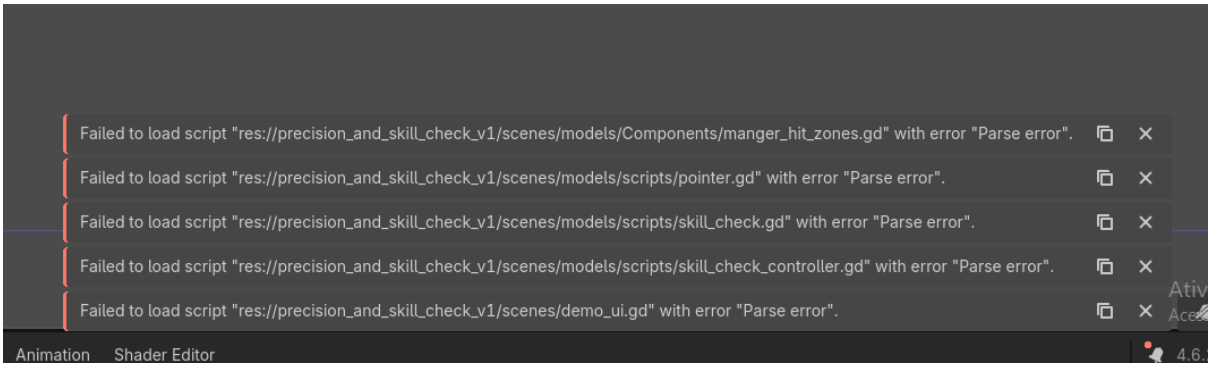
O objetivo desse recurso de Minigames de Precisão e Skill Check é fornecer uma base totalmente customizável de fácil implementação e customização, com toda a lógica de funcionamento já abstraída, com recursos direto pelo Inspetor da Engine, documentação interna ao passar o mouse por cima de alguns parâmetros

Como instalar este recurso

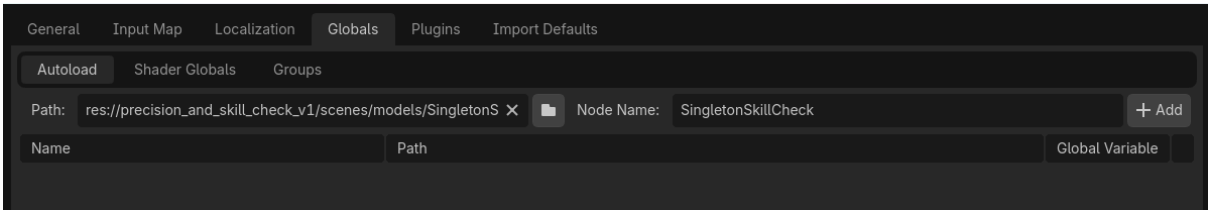
1- Extraia o arquivo ZiP Dentro de Seu Projeto



AVISO: ***Ao abrir A godot irá aparecer vários Erros, eles são normais, pois ainda precisamos Configurar o Singleton***



2- Em Project Settings Na Aba Global, Selecione o arquivo SingletonSkillCheck e coloque o mesmo nome, clique em ADD



3- Agora é só usar

“O que vou ser capaz de fazer?”

Com este recurso será capaz de fazer minigames **semelhantes(Não iguais)** aos já visto em diversos tipos de jogos.

ALERTA: *Como o foco deste recurso era ser simples e fácil o foco maior foi em abstração, por isso sons e animações ficam por conta do utilizador.*

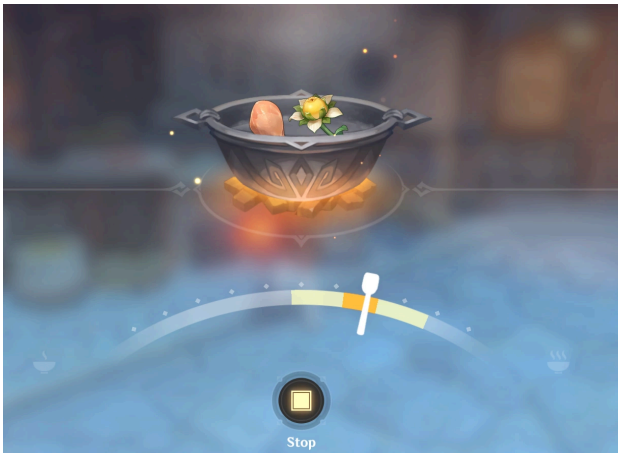
Sobre Texturas

Você vai notar que em muitos nodes o atributo “Texture” é um sprite2D, sinta-se à vontade para mudar, o objetivo era deixar o mais simples possível e customizável esse asset, apenas abstraindo toda a lógica principal para que os desenvolvedores foquem no design ou no sistema de pontuação

Inspirações & Exemplos na Indústria



Dead by Daylight - 2016



Genshin Impact - 2020



Video Horror Society (Game)



Halloween The Game - 2026

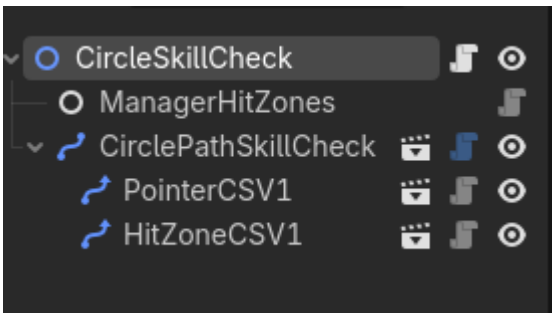


Choo-Choo Charles - 2022

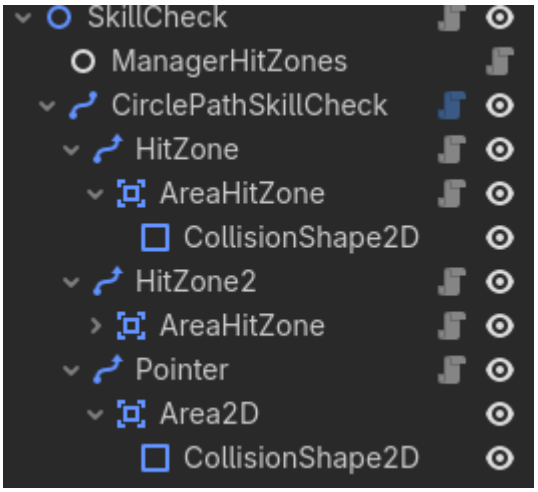
Estruturas

Packed Scenes

Confira no arquivo circle_skill_check.tscn

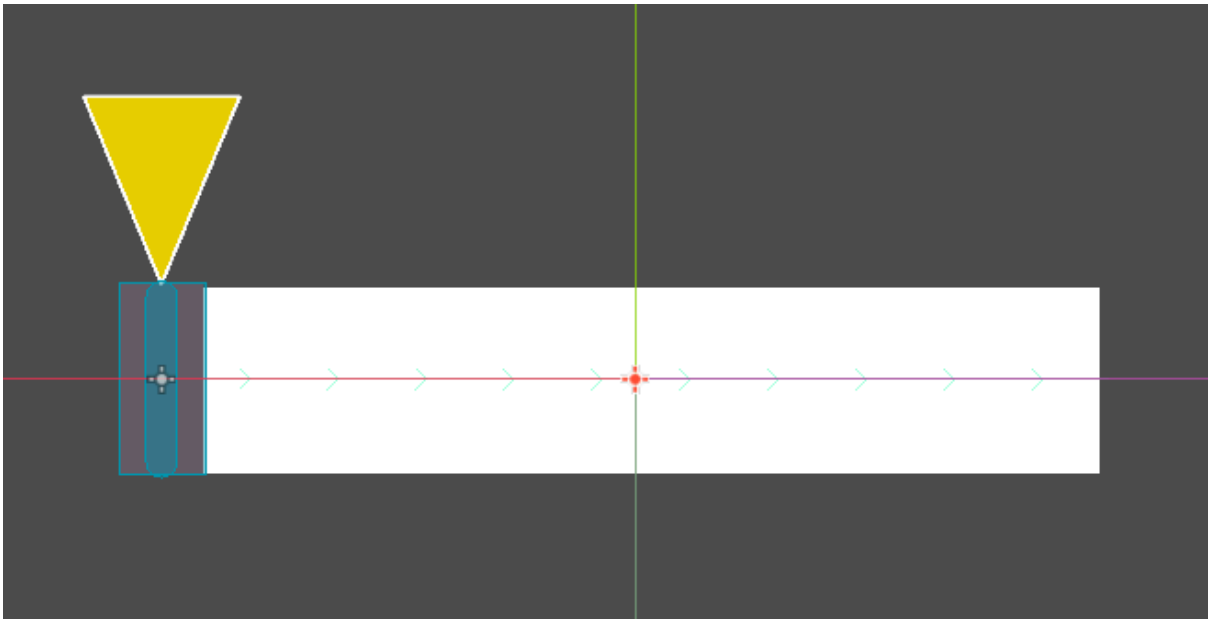


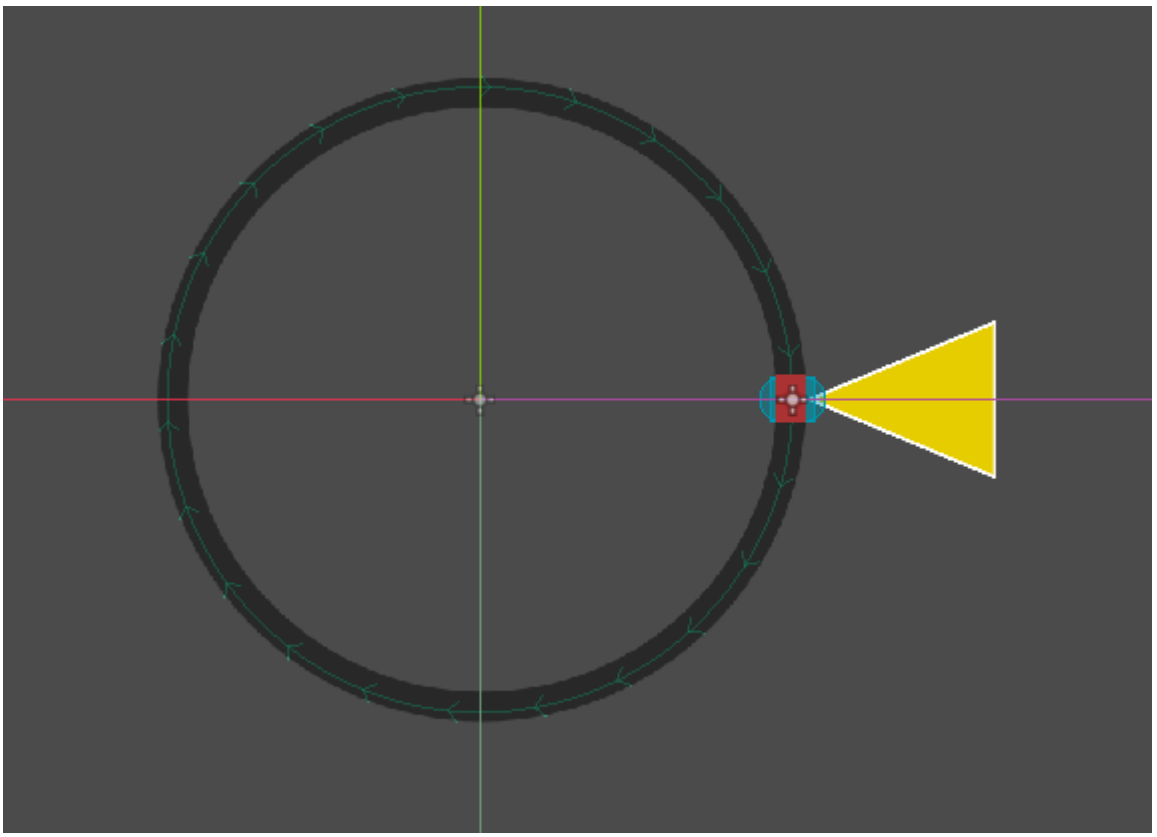
Individuais(Nodes)



***OBSV**: Texturas e sons desconsiderados*

Pré-Fabricados





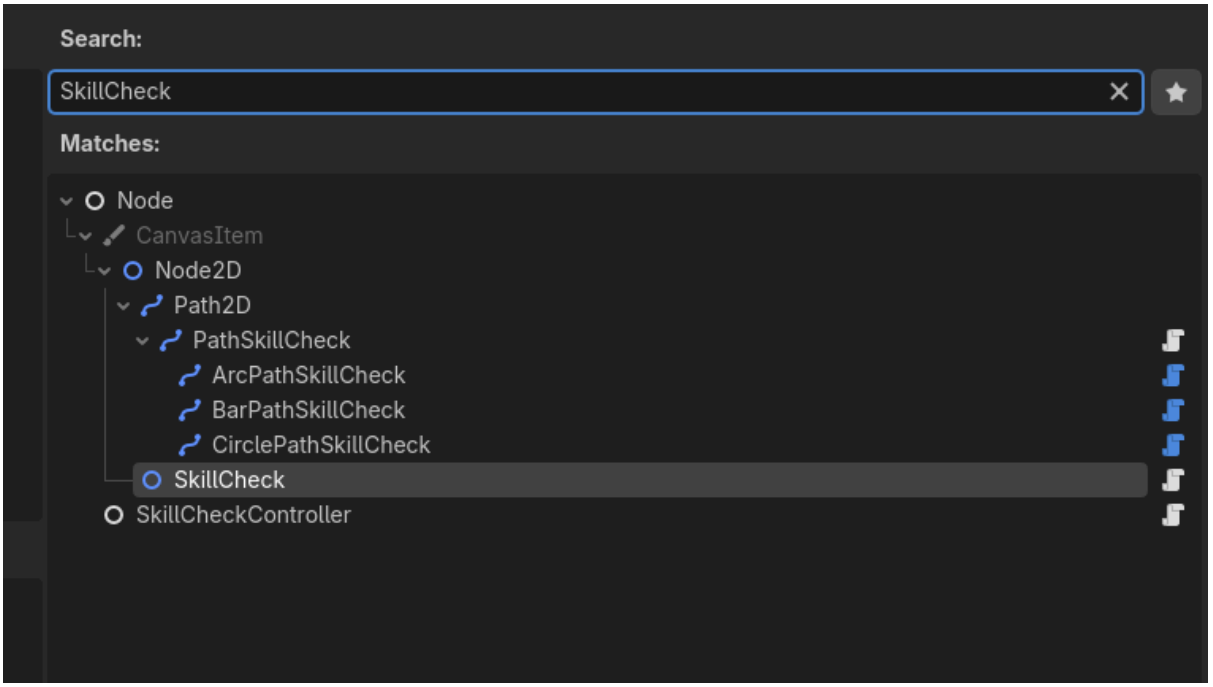
Criando Exemplo do Zero

Exemplos 1 e 2

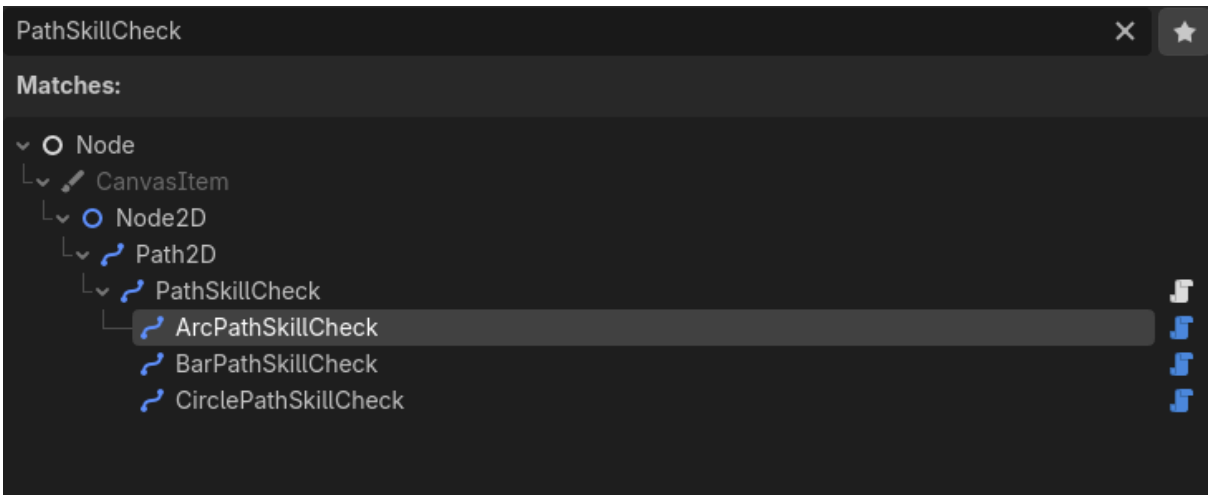
Clicando aqui você pode assistir a construção dos Exemplos 1 e 2:
https://youtu.be/012sXF4XGPo

Construindo Exemplo 3

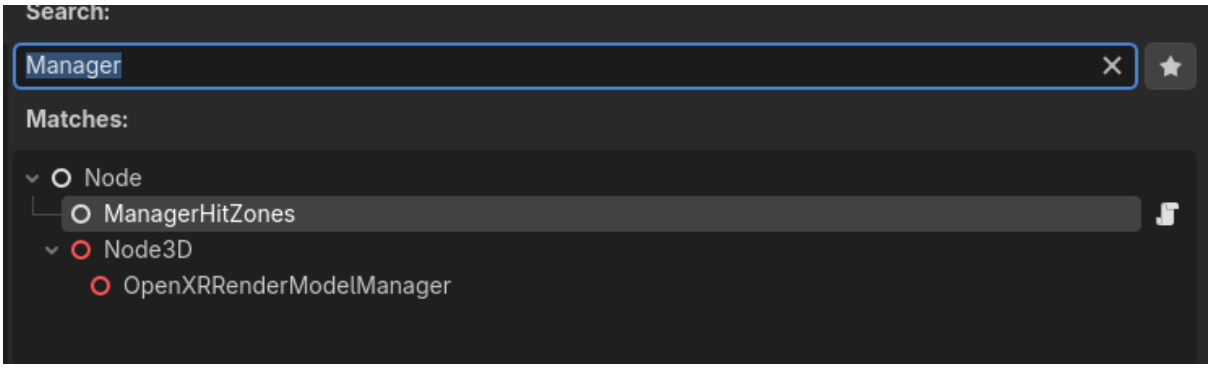
1- Adicione o Node da Classe SkillCheck



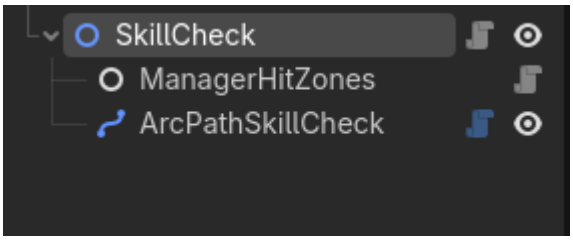
2-Adicione um Node PathSkillCheck como Filho de SkillCheck, neste caso adicionaremos o ArcPathSkillCheck, lembre-se, pode conferir a particularidade de cada classe no resto da documentação



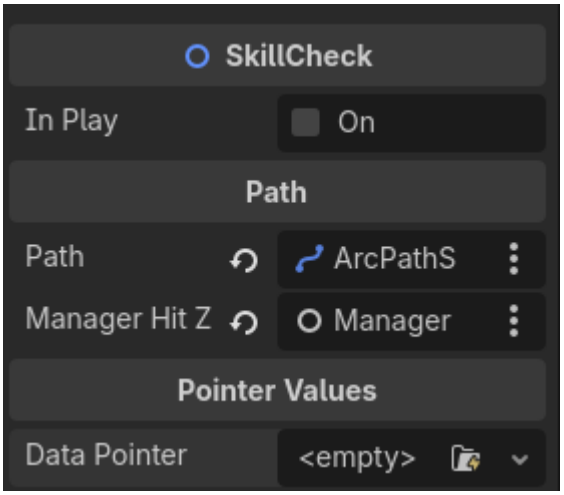
3- Adicione também o Node Manager HitZone, pode ser filho ou não do ArchPathSkillCheck



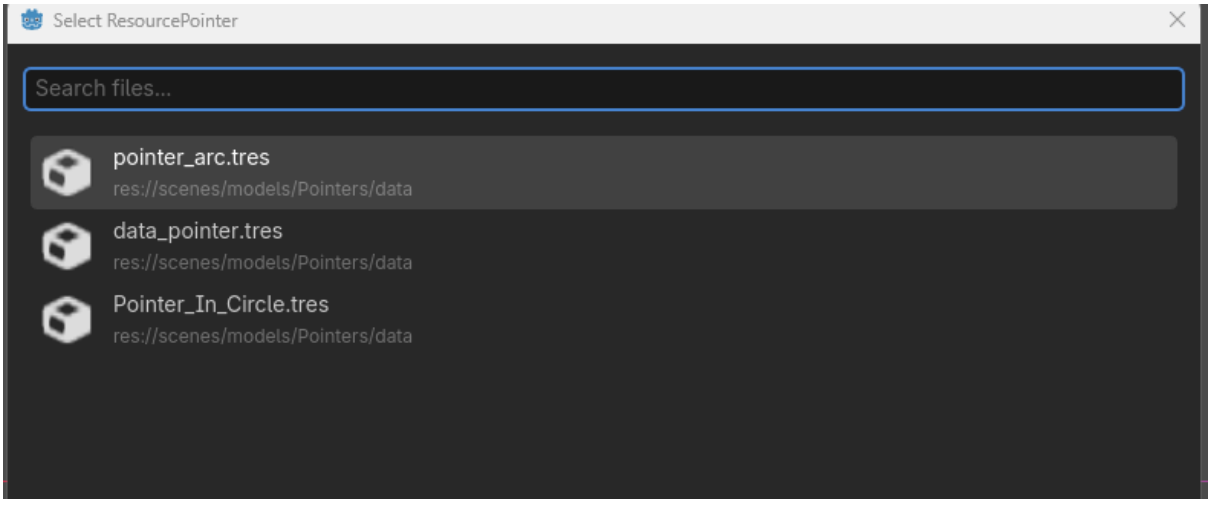
3.5-Sua estrutura deve estar mais ou menos igual a essa



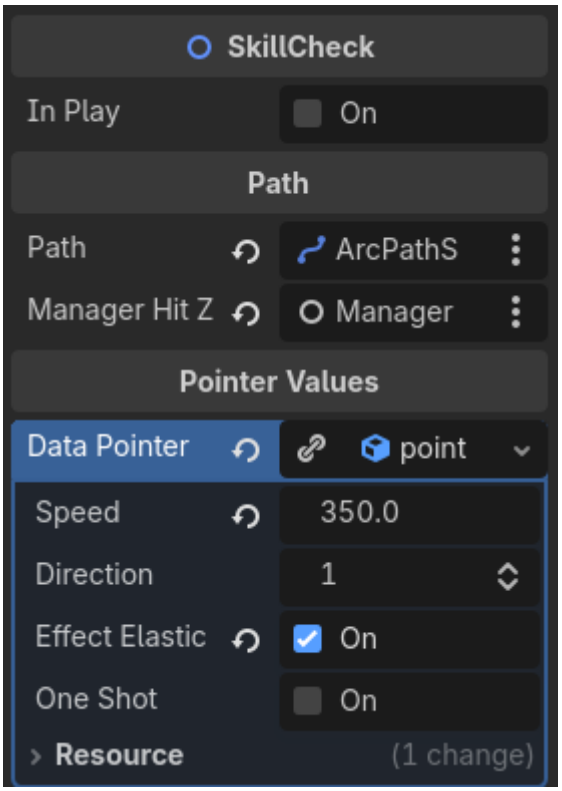
4- Com Skill Selecionado, você deve selecionar o Path e o Manager HitZone no inspetor, atribuindo os Nodes corretos



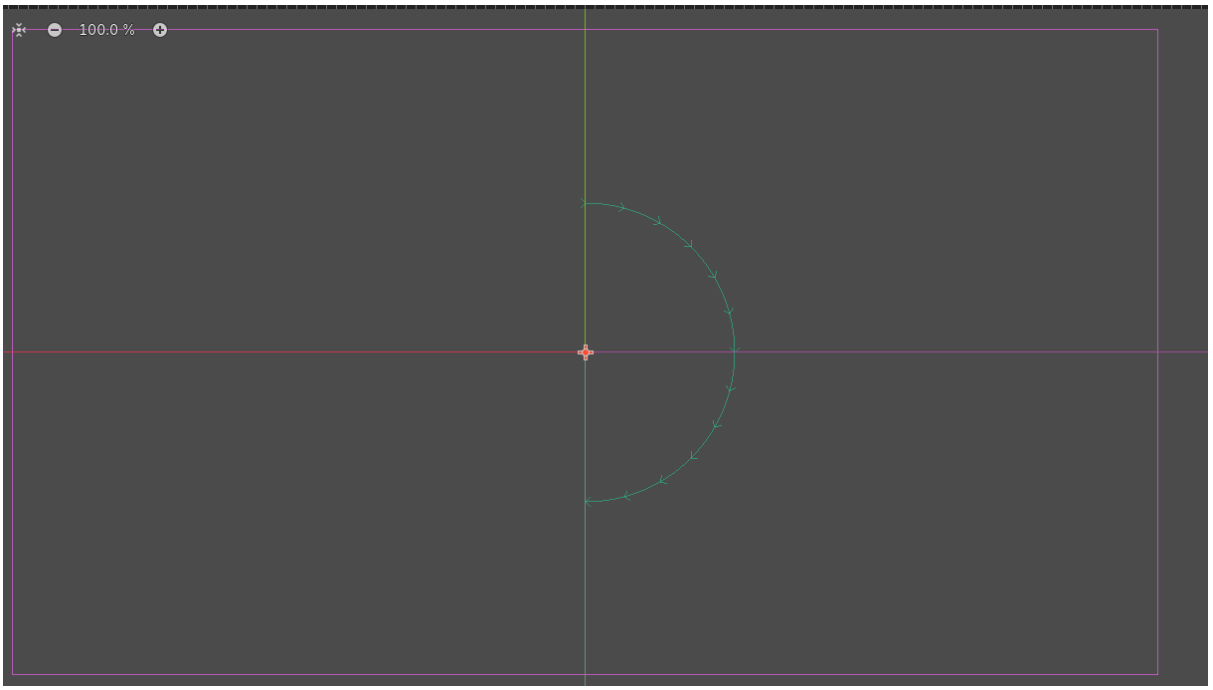
5-Em data Pointer Selecione uma das características do Ponteiro/Seletor, nese caso usaremos o pré-fabricado pointer_arc.tres



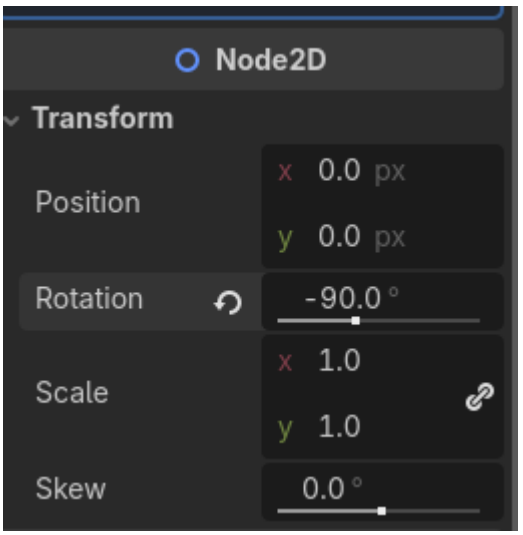
5.5- Seu skillCheck deve estar parecido com isso



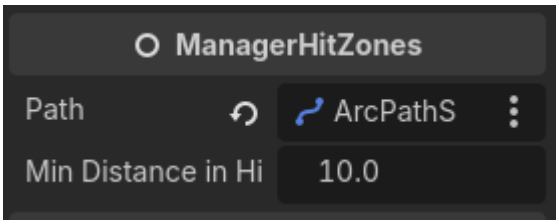
6- No inspetor seu Path vai estar assim



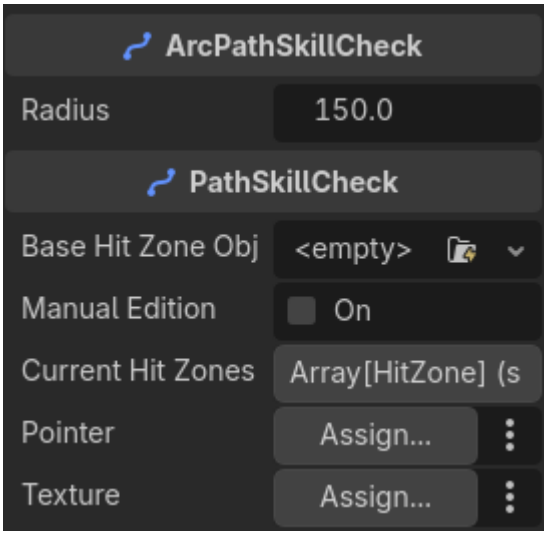
Para mudar Vá em Transform da propriedade Node2D e rotacione -90graus



7-Selecione ManagerHitZones e em seu Inspetor atribua o ArcPathSkillCheck, deve ficar assim



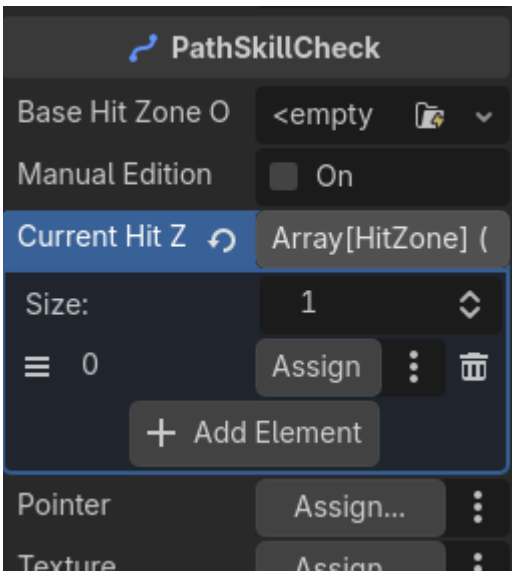
8- Selecione ArcPathSkillCheck e veja em seu Inspetor.



Radius podemos alterar o tamanho do arco a ser desenhado
Base HitZone você pode adicionar um PackedScene para spawn de HitZone dinâmicas

Manuel Edition, caso deseje alterar o path pelas ferramentas padrao do Path2D

Current Hitzone: São as HitZones atuais em seu Path, para entender melhor busque por HitZones, aqui começamos adicionando um membro que atribuímos um node depois
Pointer: Será o ponteiro/Seletor que também iremos Criar



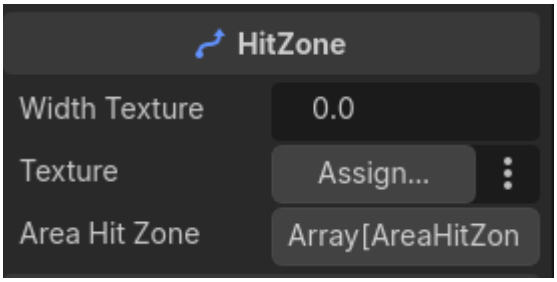
8-Adcione como filho de ArchPathSkillCheck, HitZone



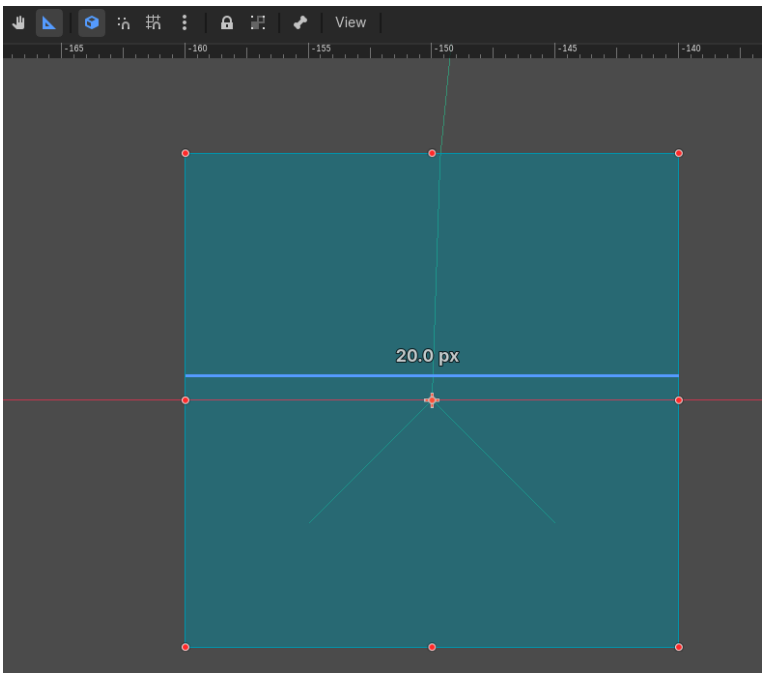
9-Adcione como filho do node HitZone, um AreaHitZone(Como herda de um Area2D precisa de um collisionShape2D



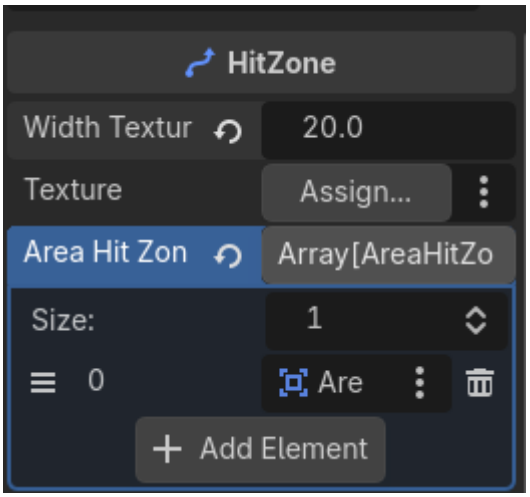
10- Selecione HitZone no Inspetor



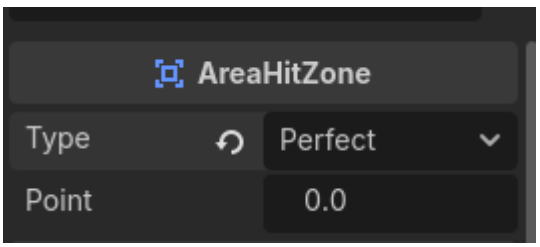
Width Texture : Coloque o valor em pixel da largura da Area/Textura, para entender melhor procure por HitZone aqui na documentação, no caso não usaremos textura então usarei a largura do Collision Shape da AreaHitZone



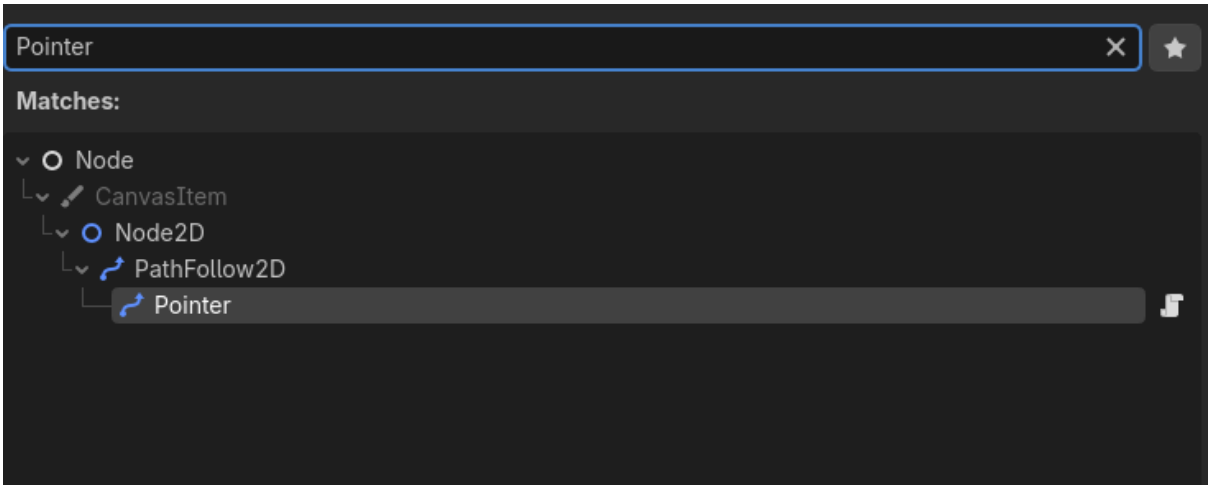
Area Hit Zone: Adicione 1 Elemento e Atribua o Node Area HitZone filho de HitZone



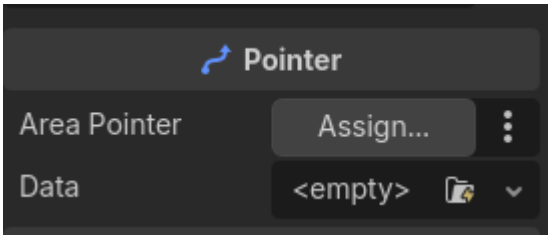
11-Selecione AreaHitzone e no inspetor selecione o tipo de Acerto dela ou pontuação



12- Selecione ArchPathSkillCheck e adcione o Node Pointer como seu filho

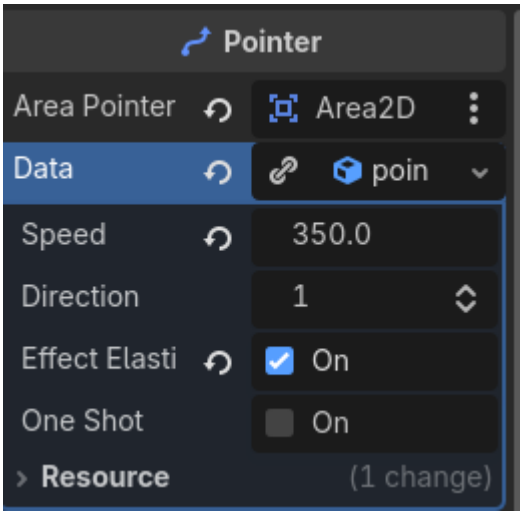


13- Note que no inspetor de Pointer



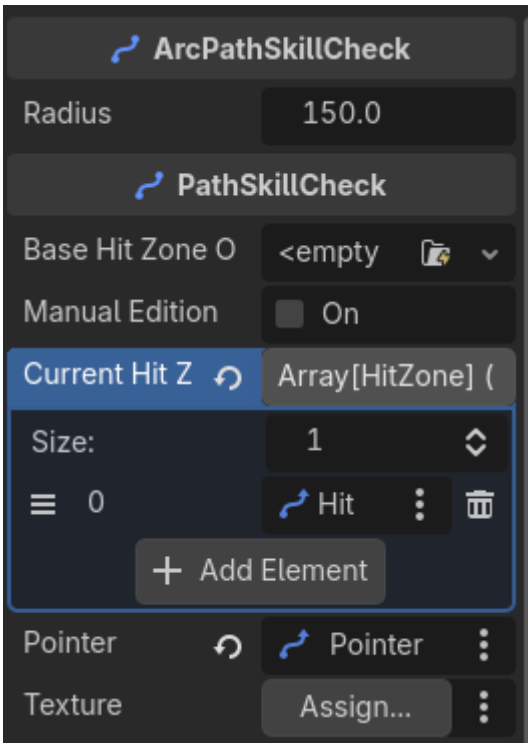
AreaPointer : Um Area2D comum, o collisionShape usado para este exemplo foi em formato cilindrico

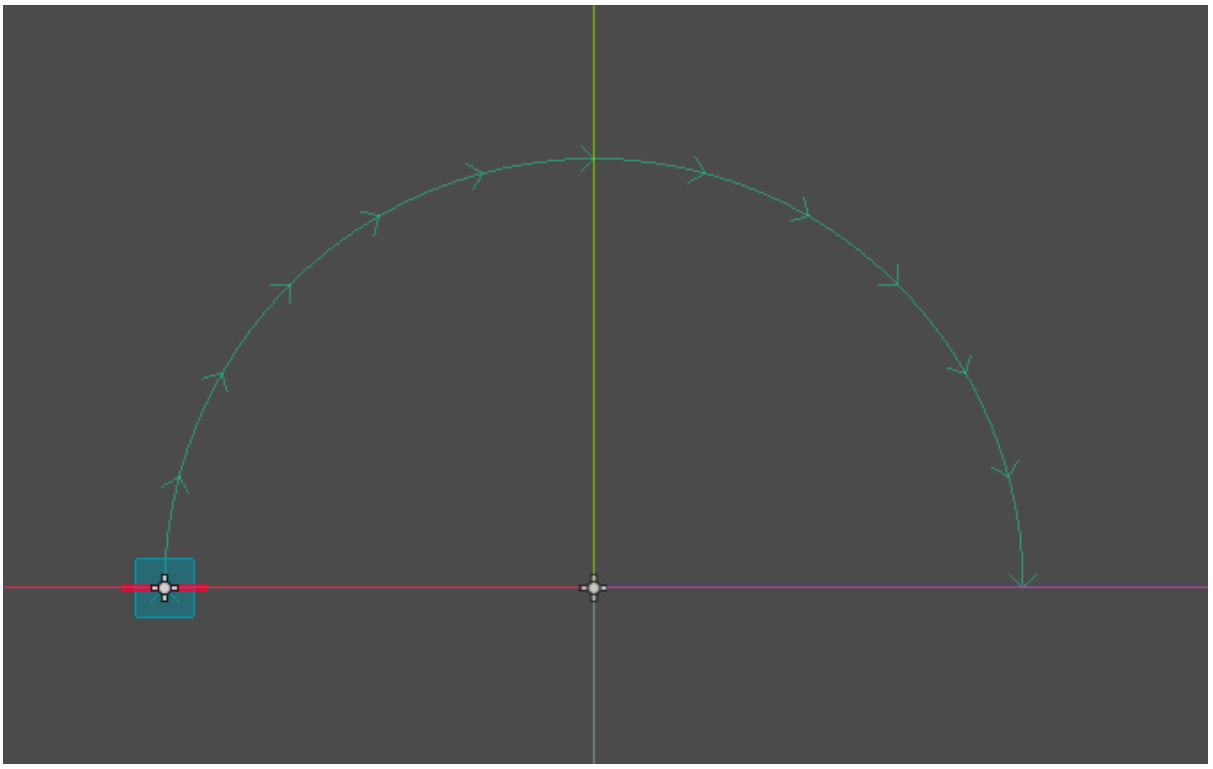
Data: As características do Pointer, deve selecionar o mesmo que escolheu em skillCheck



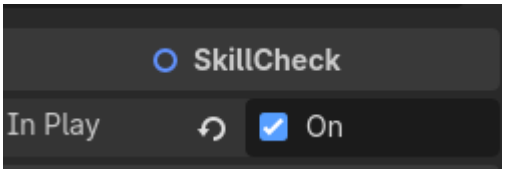
No caso o arquivo : pointer_arc.tres

14- Volte em ArcSkillCheck e realize as atribuições de HitZone e pointer

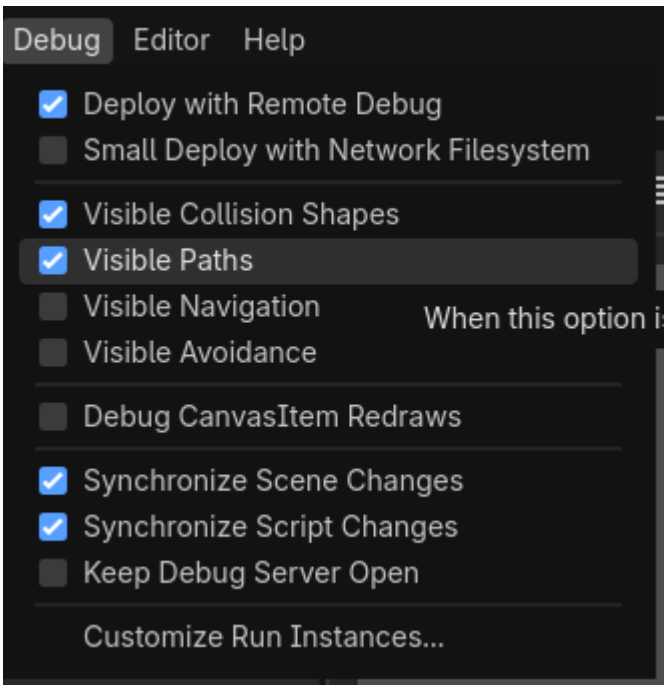




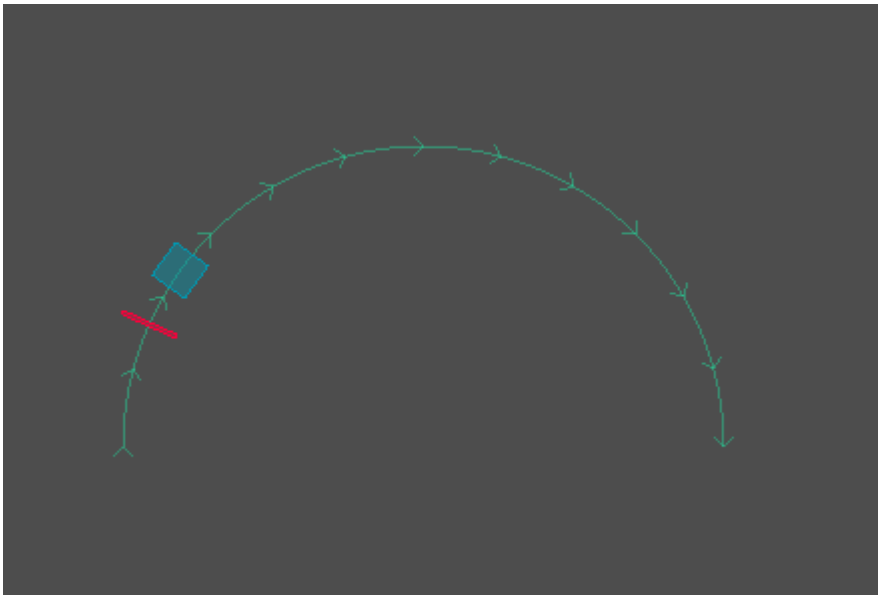
15- Para testar a Cena é preciso que In Play em SkillCheck esteja marcado como true



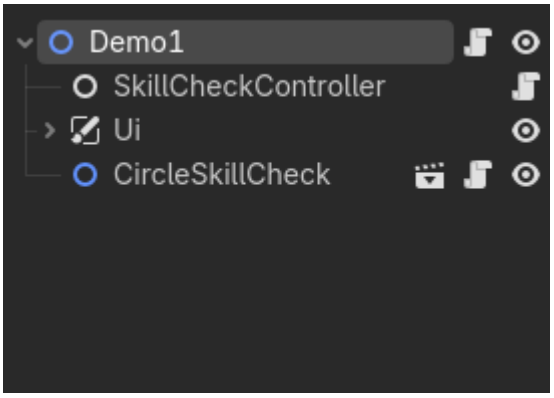
Para ver funcionar nesse caso sem textura habilite as opções em Debug



16- Deverá ficar assim ao executar a Cena



17- Veja, explore e Estude os arquivos de Demo, Scene e Scene-Prefabs



18- Leia sobre SkillCheckController

SingletonSkillCheck (AutoLoad)

Como o nome já presume, este é um script em comum e um autoload do recurso, nele encontramos os sinais emitidos durante o minigame em execução, desta maneira basta chamá-lo em qualquer script seu que deseje que faça alguma coisa quando um sinal é emitido.
Para implementar Animações e Sons considere nodes e Scripts a parte conectados aos Signals do AutoLoad

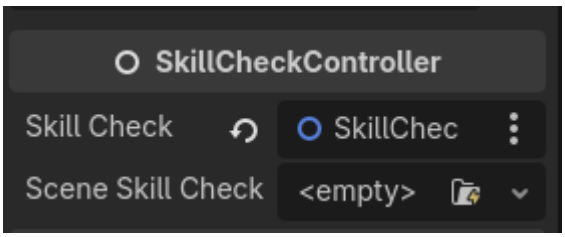
Signals

signal hit_registred(area : AreaHitZone, type : String, point : float)
Emitido quando acontece um acerto em um área da HitZone.
Ele informa(Passa como argumento) Qual área : AreaHitZone foi, seu tipo: (Anyway, Great or Perfect) e valor de pontuação : float
signal miss
Emitido quando o jogador erra, não acerta uma hitzone
signal end()
Emitido quando o Minigame termina

SkillCheckController

Esté Node é um intermediários para o controle do Minigame através de outro Script durante a execução do jogo, por exemplo, se deseja passar para o jogador um tutorial antes ou alguma cinematic

Inspetor



@export var **_skill_check** : SkillCheck
Referência ao Minigame
@export var **_scene_skill_check_model** : PackedScene
Cena salva do Minigame para ser executado com método spawn_my_skill_check(_scene_skill_check_model : PackedScene)

Métodos

_ready()

```
func _ready() -> void:
	SingletonSkillCheck.connect("hit_registred", hit_process)
	SingletonSkillCheck.connect("hit_registred", func(area : AreaHitZone, type : String, point : float):
		get_hit_zones_count())
	SingletonSkillCheck.connect("miss",miss_process)
	SingletonSkillCheck.connect("end",end_process)
```

play_skill_check():

```
func play_skill_check():
	if !_skill_check:
		print_debug("Null Skill Check")
		return
	_skill_check._play_or_pause(true)
```

pause_skill_check():

```
func pause_skill_check():
	_skill_check._play_or_pause(false)
```

Métodos conectados aos Sinais do AutoLoad SingletonSkillCheck

Esses métodos definem o que o SkillCheckController em caso dos sinais serem emitidos

```
func hit_process(area : AreaHitZone, type : String, point : float):
	pass
func miss_process():
	pass
func end_process():
	pass
```

spawn_my_skill_check(_scene_skill_check : PackedScene):

```
func spawn_my_skill_check(_scene_skill_check : PackedScene):
    var new_skill_check : SkillCheck = _scene_skill_check_model.instantiate()
    add_child(new_skill_check)
    _skill_check = new_skill_check
```

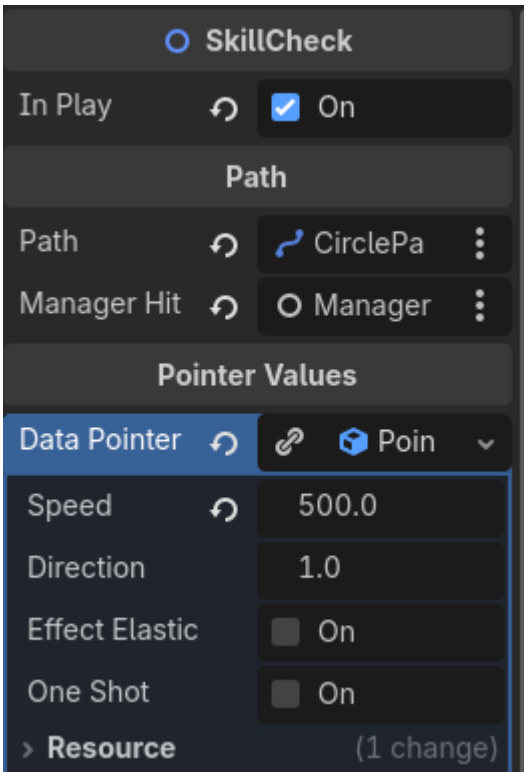
get_hit_zones_count():

```
func get_hit_zones_count():
    _skill_check.get_hit_zone_count()
    pass
```

Skill Check

SkillCheck é a classe da lógica pelo funcionamento do Minigame enquanto ele está rodando.

Inspector



In Play: Atributo inicial para definir se o minigame é jogável após o carregamento.
Path: Referência ao nó PathSkillCheck usado para o layout do caminho e o posicionamento espacial.
Manager Hitzones: Referência ao Nó ManagerHitZones para regras de organização das hitzones

Data Pointer: Referência ao ResourcePointer, alteração dos atributos diretamente pelo SkillCheck no inspector que o Pointer usará. Para entender melhor procure pelo tópico “Pointer”

Métodos

_physics_process()

Verificação do input do jogador na mecânica principal durante a execução do Minigame

```
func _physics_process(_delta: float) -> void:
    #Core gameplay input check during physics execution
    if Input.is_action_just_pressed("ui_accept"):
        if _pointer.get_hittable():
            var current_hit_zone : AreaHitZone = _pointer._current_hit_zone
            SingletonSkillCheck.hit_registred.emit(current_hit_zone, current_hit_zone.get_type(), current_hit_zone.point
        else:
            SingletonSkillCheck.miss.emit()
```

_play_or_pause(play : bool)

Definir o pause somente para o minigame

```
func _play_or_pause(play : bool):
    if !play:
        set_physics_process(play)
        _pointer.set_physics_process(play)
    else:
        set_physics_process(play)
        _pointer.set_physics_process(play)
```

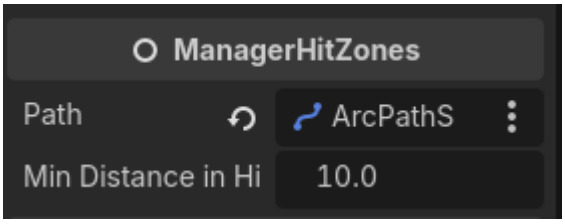
get_hit_zone_count()

Verifica se ainda Há HitZones no PathSkillCheck, caso não tenha o Minigame Emite o Sinal de “End”

```
func get_hit_zone_count():
    if path.get_hit_zones().size() <=0:
        SingletonSkillCheck.end.emit()
```

Manager HitZones

Este componente é responsável por organizar a distribuição das HitZones no “PathSkillCheck”(O caminho que o ponteiro irá percorrer).



Path : Pelo Inspector selecione o PathSkillCheck que o componente deverá organizar, caso contrário o componente não irá organizar as HitZones

Min Distance in HitZones : É um valor Float, definido em pixel da distância mínima entre as hitzones caso tenha mais de uma HitZone.

Métodos

_ready()

```gdscript
func _ready() -> void:
    path.connect("new_hit_zones",update)
    var hit_zones : Array[HitZone] = path.get_hit_zones()
    var size_path : float = path.get_size_path()
    organize_hit_zones(hit_zones,size_path, min_distance_in_hit_zones)
```

organize_hit_zones()

```gdscript
func organize_hit_zones(hit_zones: Array[HitZone],size_path: float, min_distance: float = 10.0) -> void:
    var total_zones = hit_zones.size()
    if total_zones == 0:
        return
    if  total_zones == 1:
        var hit_zone = hit_zones[0]
        var min_path : float = 0 + (hit_zone.get_width_texture()/2)
        var max_path : float = (size_path - ( hit_zone.get_width_texture()/2))
        hit_zone.progress = randf_range(min_path,max_path)
        return
    var total_length: float = path.get_size_path()
    var texture_width: float = hit_zones[0].get_width_texture()
    # Minimum required distance between the centers of two HitZones
    var required_separation: float = texture_width + min_distance
    # Validate if the path has enough total space
    if (required_separation * total_zones) > total_length:
        push_warning("The path is too short to fit all HitZones with this minimum clearance!")
    var placed_positions: Array[float] = []
    for i in range(total_zones):
        var position_found: bool = false

        # Attempts to pick a valid random position within attempt limit
        for attempt in range(min_distance):
            # Pick a progress value keeping margins away from path edges
            var candidate_progress: float = randf_range(texture_width / 2.0, total_length - (texture_width / 2.0))
            var valid: bool = true

            # Check separation against already placed zones
            for pos in placed_positions:
                if abs(candidate_progress - pos) < required_separation:
                    valid = false
                    break

            if valid:
                placed_positions.append(candidate_progress)
                hit_zones[i].progress = candidate_progress
                position_found = true
                break

        # Fallback if random sampling fails to find free space
        if not position_found:
            push_warning("Could not find a valid random position without overlap for HitZone %d." % i)
            var fallback_progress: float = (i * required_separation) + (texture_width / 2.0)
            hit_zones[i].progress = clampf(fallback_progress, 0.0, total_length)
```

update()

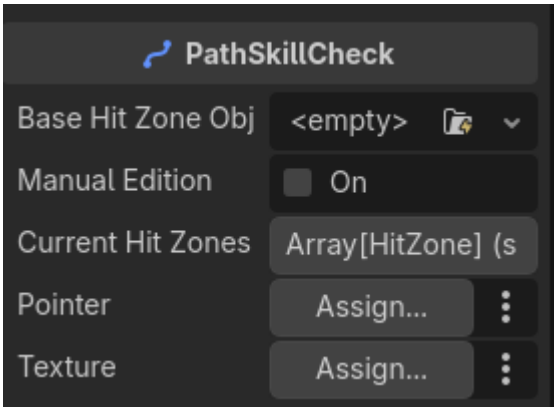
```gdscript
func update():
    var hit_zones : Array[HitZone] = path.get_hit_zones()
    var size_path : float = path.get_size_path()
    organize_hit_zones(hit_zones,size_path,min_distance_in_hit_zones)
```

PathSkillCheck

Componente de SkillCheck que define a trajetória Path2D e oferece suporte de layout para HitZones e Pointer/Seletor.Serve como base espacial para minigames baseados em trajetórias, fornecendo a geometria do caminho e o posicionamento espacial para indicadores e zonas de acerto.

Estenda esta classe para implementar geometrias de trajetórias específicas por exemplo, CirclePathSkillCheck ou ArcPathSkillCheck

O fluxo geral do minigame, a avaliação de acertos e a lógica de ciclo de vida são gerenciados pela classe pai SkillCheck.



Base Hitzone Object : Tipo de cena padrão usado para instanciar automaticamente uma HitZone.
Manual Edition: Se **true** o caminho deve ser editado manualmente ; Se **false**, ele deve seguir regras definidas via **rules_build_path().**
Current Hit Zones: Lista de HitZones associadas a este caminho.
Importante: *Sempre adicione suas zonas de acerto a esta lista, mesmo que haja apenas uma.*
Pointer: Referência ao Pointer carregado durante o método _ready().

Métodos

rules_build_path()

Se estiver vazio, o caminho deve ser desenhado por meio do Inspetor, utilizando a propriedade herdada Path2D.

```
func rules_build_path() -> void:
    pass
```

get_size_path() -> float

```
func get_size_path() -> float:
    return curve.get_baked_length()
```

get_hit_zones()-> Array:

```
func get_hit_zones()-> Array:
    return _current_hit_zones
```

get_pointer() -> Pointer

```
func get_pointer():
    return pointer
```

add_hit_zones()

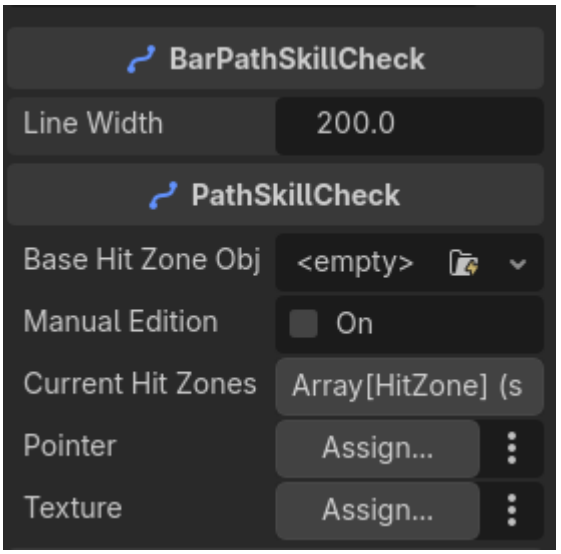
```
func add_hit_zones( amount : int = 1,object_scene : PackedScene = base_hit_zone_Object_Scene ):
    for new_hitzones in amount:
        var new_instantiate : HitZone = object_scene.instantiate()
        _current_hit_zones.append(new_instantiate)
        new_hit_zones.emit()
```

Signal

new_hit_zones

Emitido quando uma nova Hitzone é adicionada por meio do método “add_hit_zones()”

BarPathSkillCheck



Line Width : Recebe um valor float, desenha e exibe no inspetor uma linha

Métodos

```gdscript
func rules_build_path():
    build_horizontal_line_path(line_width)

func build_horizontal_line_path(length: float) -> void:
    var new_curve: Curve2D = Curve2D.new()
    var half_length: float = length / 2.0

    # Ponto 1: Extremidade Esquerda
    new_curve.add_point(Vector2(-half_length, 0.0))

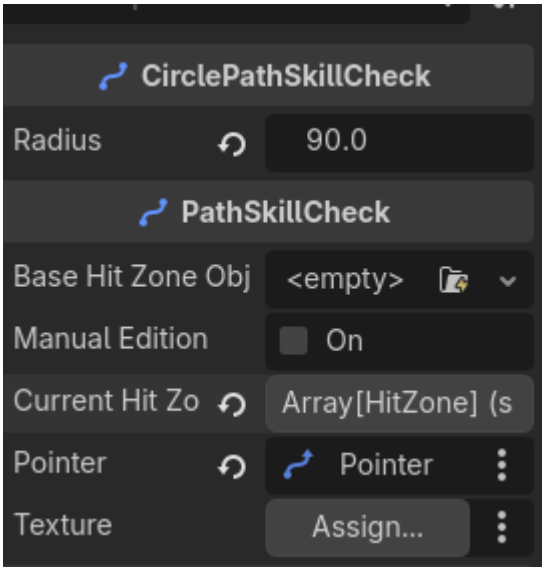
    # Ponto 2: Extremidade Direita
    new_curve.add_point(Vector2(half_length, 0.0))

    curve = new_curve
```

CirclePathSkillCheck

Herda as características de PathSkillCheck, além de adicionar desenho automático de um círculo definido pelo valor do raio em seu inspetor, ao alterar o valor de “Radius” automaticamente no inspetor um novo círculo é desenhado.

Inspetor



Radius : Recebe um valor float, desenha e exibe no inspetor o novo círculo desenhado

Métodos

```gdscript
func rules_build_path() -> void:
	create_circle_path()
	pass

## Generates a perfect circular path using Path2D curves.
func create_circle_path():
	var new_curve = Curve2D.new()
	var points = 4 #4 cardinal points
	var handle_length = radius * 0.5522847498 # Constant for optimal Bézier circle approximation

	for i in range(points):
		var angle = i * (PI / 2.0)
		var pos = Vector2(cos(angle), sin(angle)) * radius

		#Control vectors (Handles)
		var dir_in = Vector2(sin(angle), -cos(angle)) * handle_length
		var dir_out = -dir_in

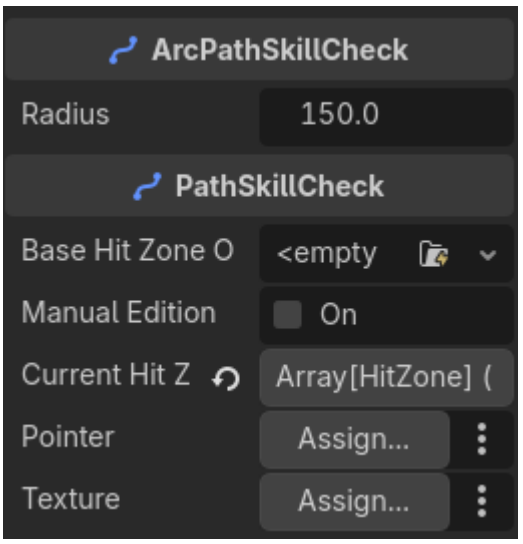
		new_curve.add_point(pos, dir_in, dir_out)

	# Closes the circle by connecting the last point back to the first
	new_curve.add_point(new_curve.get_point_position(0), new_curve.get_point_in(0), new_curve.get_point_out(0))
	curve = new_curve
```

ArcPathSkillCheck

Herda as características de PathSkillCheck, além de adicionar desenho automático de um Arco definido pelo valor do raio em seu inspetor, ao alterar o valor de “Radius” automaticamente no inspetor um novo Arco é desenhado.

Inspetor



Radius : Recebe um valor float, desenha e exibe no inspetor o novo Arco desenhado

Métodos

```gdscript
func rules_build_path() -> void:
    create_arc()

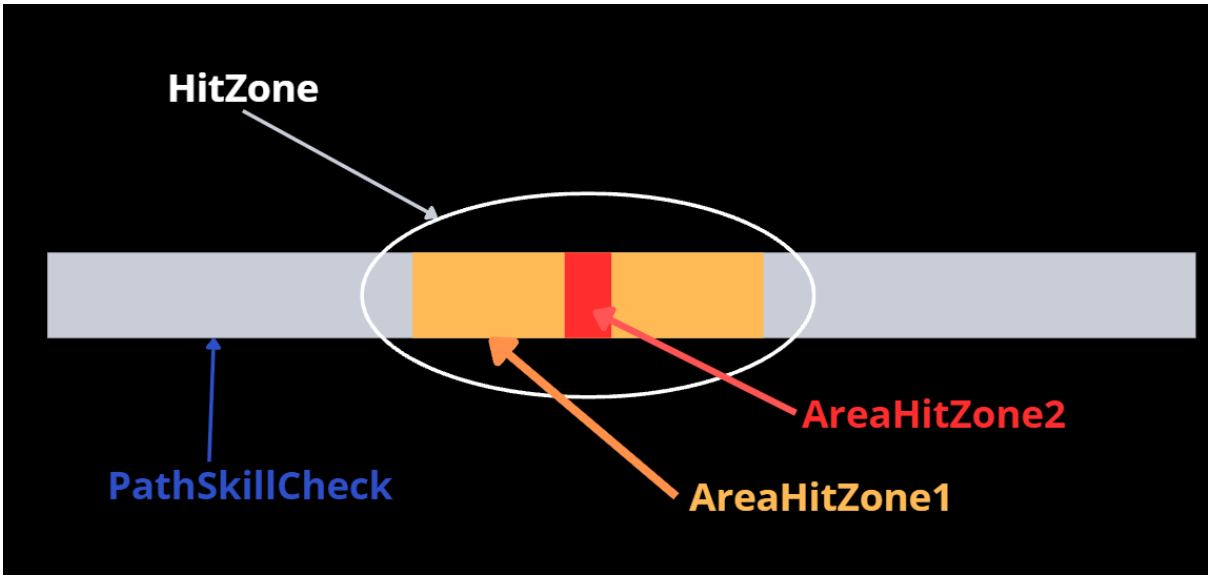
func create_arc():
    var new_curve: Curve2D = Curve2D.new()
    var handle_length: float = radius * 0.5522847

    # Ponto 1: Topo
    new_curve.add_point(Vector2(0, -radius), Vector2.ZERO, Vector2(handle_length, 0))
    # Ponto 2: Vértice Curvo (Direita)
    new_curve.add_point(Vector2(radius, 0), Vector2(0, -handle_length), Vector2(0, handle_length))
    # Ponto 3: Base
    new_curve.add_point(Vector2(0, radius), Vector2(handle_length, 0), Vector2.ZERO)

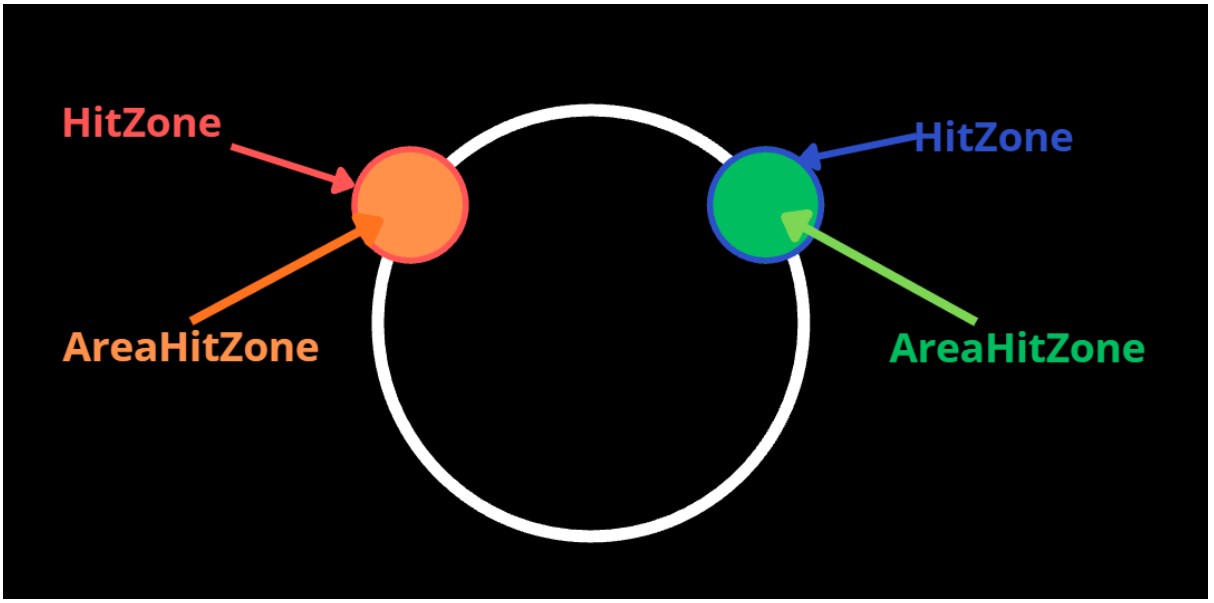
    curve = new_curve
```

HitZone

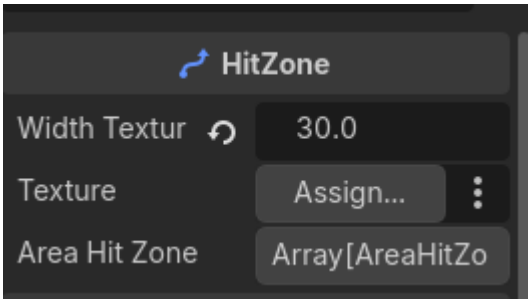
Hitzone representa área de acerto em um PathSkillCheck. Uma hitZone pode contar várias AreaHitZone com significados ou valores diferentes.Muito comum nesse tipo de minigame ter uma área ao lado da outra que pode significar “Perfect” ou “Great”.
Pode-se entender que HitZone é formada por 1 ou mais AreaHitZone.
Exemplos:
1- Uma HitZone com duas AreaHitZone



2- Duas HitZones com uma AreaHitZone cada



Inspetor



Width Texture : Referência manual de largura da textura usada para os cálculos de layout da Hitzone;Devido a variações nos métodos de definição de tamanho de formas na API do Godot,este valor simplifica os cálculos de distribuição e posicionamento da zona de acerto. Esta propriedade é somente leitura para fins de cálculo de layout e não redimensiona nem escala a textura em si.
Nota: Caso tenha mais de uma AreaHitZone, deve-se colocar o valor das largura somados
***Importante* : Pode-se usar o valor da Altura invés da largura, o que se adaptar melhor em seu path**

Texture : Textura de sprite visual usada como fundo para a área-alvo da HitZone.
Serve como fundo quando existem múltiplas regiões de acerto dentro de uma única zona de acerto;
Caso contrário, pode ser usada para referenciar diretamente o sprite principal da zona de acerto.
Area Hit Zone : Lista responsavel por armazenar as áreas da HitZone, uma Hitzone pode ter várias áreas com valores diferentes de pontuação e tipo de acerto

Métodos

get_width_texture() -> float

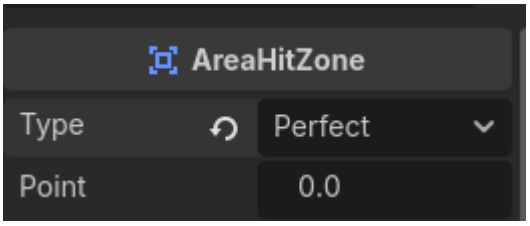
```
func get_width_texture():
    return _width_texture
```

get_hit_area() -> Array

```
func get_hit_area():
    return _area_hit_zone
```

AreaHitZone

Inspetor



Type: Escolha Entre “Anyway, Great e Perfect”
Point: Valor Float de quantos pontos representa a área

Métodos

get_type()->String:

```
## Returns the string representation of the hit accuracy type
func get_type()-> String:
    var name_type : String = Sucess.keys()[_type]
    return name_type
```

get_size_shape():

Método que retorna o tamanho do collision shape, caso seja uma retângulo ou quadrado, para pegar de outra forma existem outros métodos específicos

```
func get_size_shape():
    return _collision_shape.shape.size
```

hittable():

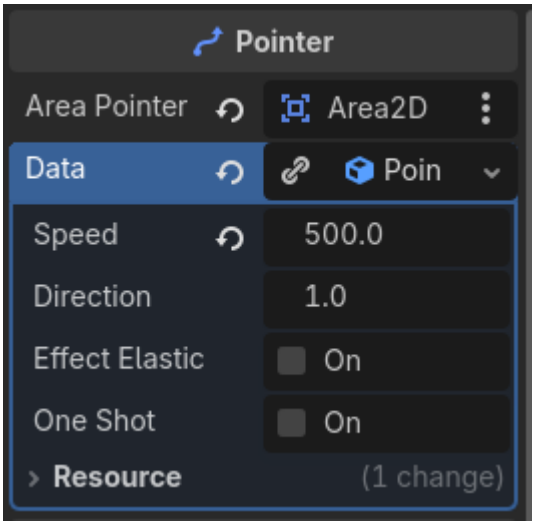
Nesse bloco de código o que acontece quando a Área leva um HIT

```
func hittable():
    queue_free()
```

Pointer

Pointer ou Seletor é basicamente a “Agulha” que percorre todo o Caminho(PathSkillCheck), e dependendo onde ela estiver quando o usuário apertar a ação de “Hit” terá o resultado se acertou ou não uma HitZone.

Inspetor



Área Pointer : É uma Area2D comum, apenas para funcionamento da detecção das HitZones.

Data: Recebe um ResourcePointer, pode ser criado pelo inspetor, mas recomendo a utilização de um pré-configurado ou salvo.

Speed: A velocidade com que se move pelo Path.

Direction: A direção em que começa.

Effect Elastic: Quando chegar ao final do Path irá fazer o caminho inverso(diferente do atributo loop do PathFollow2D).

One Shot: Irá parar de se mover ao percorrer o Path por completo uma vez.

Métodos

_ready():

```
func _ready():
    if data.one_shot && data.effect_elastic:
        data.effect_elastic = false
    loop = !data.effect_elastic
    speed_pointer = data.speed
    pointer_direction = data.direction
    _area_pointer.area_entered.connect(_can_hit)
    _area_pointer.area_exited.connect(not_hit)
```

_physics_process(delta) and move_pointer(delta)

```
func _physics_process(delta: float) -> void:
    move_pointer(delta)

func move_pointer( delta):
    progress += speed_pointer * pointer_direction * delta

    if loop and data.one_shot:
        if progress_ratio >= 0.9:
            progress_ratio = 1.0
            SingletonSkillCheck.end.emit()
            set_physics_process(false)
        return
    else:
        if progress_ratio >= 1.0:
            progress_ratio = 1.0
            pointer_direction = -1.0
        elif progress_ratio <= 0.0:
            progress_ratio = 0.0
            pointer_direction = 1.0
```

_is_hit_zone(area : Area2D)

```
func is_hit_zone(area : Area2D) -> bool:
    if area is AreaHitZone:
        return true
    else:
        return false
```

_can_hit(area : Area2D)

```
func _can_hit(area : Area2D):
    var i_can : bool = is_hit_zone(area)
    if !i_can:
        return
    else:
        hittable = true
        _current_hit_zone = area
```

not_hit(area : Area2D)

```
func not_hit(area : Area2D):
    var i_not : bool = is_hit_zone(area)
    if !i_not:
        return
    else:
        hittable = false
        _current_hit_zone = null
```

getters

```
func get_speed_pointer():
    return speed_pointer
func get_hittable():
    return hittable
func get_direction():
    return pointer_direction
```

ResourcePointer

Atributos

@export var **speed** : float

@export var **direction** : int

@export var **effect_elastic** : bool

Alterna o efeito de rebote elástico para o movimento do ponteiro.
Quando ativado, o loop de caminho é desativado automaticamente.
Deve permanecer desativado em caminhos circulares completos

@export var **one_shot** : bool

Quando ativado, o minigame termina automaticamente após o ponteiro completar um único percurso.
*Não pode ser usado em conjunto com **effect_elastic**.*

Licença

Este Recurso é gratuito sob a **licença MIT. Você pode usá-lo em projetos comerciais e pessoais. A atribuição é obrigatória**. Por favor, inclua os créditos no seu Projeto da seguinte forma:
"Template de “*ShakerCTRL”* (Link do seu GitHub/Itch.io)"

Conclusão

Esse Asset foi um desafio que dei a mim mesmo de tentar construir algo com meus conhecimentos da Engine e de Desenvolvimento de Software.Inteligência artificial só foi usada para cálculos matemáticos,e para a tradução do idioma **Inglês**, toda a estrutura e lógica de funcionamento foi desenvolvida por mim, sei que não está perfeito e das limitações fora a maneira de instalar, fiz o melhor que pude e pretendo melhorar sua estrutura no futuro ou transformá-la em um Plugin, mas se você se dispôs um pouco para ler fico muito Agradecido!

Siga nas redes Sociais

@ShakerCTRL